Practical No.8 (Project)

Topic: Course Registration System

GUI Code:

LoginScreen code:

```
import javax.swing.*;

import javax.swing.border.EmptyBorder;

import java.awt.*;

import java.awt.event.ActionEvent;

import java.awt.event.ActionListener;

import java.sql.Connection;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.sql.SQLException;

public class LoginScreen extends JFrame {

    private JTextField usernameField;

    private JPasswordField passwordField;

    private JButton loginButton;

    private JComboBox<String> roleComboBox;

    public LoginScreen() {

        setTitle("Elective Course Registration System - Login");

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setSize(600, 400);

        setLocationRelativeTo(null); // Center the frame

        // Background Image

        JLabel backgroundLabel = new JLabel(new ImageIcon("img2.jpg"));

        backgroundLabel.setLayout(new BorderLayout()); // Use BorderLayout for main panel
positioning
```

```
// Transparent Main Panel
JPanel mainPanel = new JPanel(new GridLayout(5, 2, 10, 10));
mainPanel.setOpaque(false); // Make the panel transparent
mainPanel.setBorder(new EmptyBorder(30, 30, 30, 30)); // Padding around the panel
// Title JLabel
JLabel titleLabel = new JLabel("Elective Course Registration System", JLabel.CENTER);
titleLabel.setForeground(Color.DARK_GRAY);
titleLabel.setFont(new Font("Arial", Font.BOLD, 14));
// UI Components
JLabel roleLabel = new JLabel("Role:");
roleLabel.setForeground(Color.BLACK);
roleLabel.setFont(new Font("Arial", Font.BOLD, 16));
roleComboBox = new JComboBox<>(new String[]{"Admin", "Student"});
roleComboBox.setBackground(new Color(230, 230, 230)); // Light background
roleComboBox.setFont(new Font("Arial", Font.PLAIN, 16));
roleComboBox.setPreferredSize(new Dimension(150, 30));
JLabel usernameLabel = new JLabel("Username:");
usernameLabel.setForeground(Color.BLACK);
usernameLabel.setFont(new Font("Arial", Font.BOLD, 16));
usernameField = new JTextField();
usernameField.setBackground(new Color(230, 230, 230));
usernameField.setFont(new Font("Arial", Font.PLAIN, 16));
usernameField.setBorder(BorderFactory.createLineBorder(new Color(0, 153, 204), 1));
JLabel passwordLabel = new JLabel("Password:");
passwordLabel.setForeground(Color.BLACK);
passwordLabel.setFont(new Font("Arial", Font.BOLD, 16));
passwordField = new JPasswordField();
passwordField.setBackground(new Color(230, 230, 230));
passwordField.setFont(new Font("Arial", Font.PLAIN, 16));
```

```java
    passwordField.setBorder(BorderFactory.createLineBorder(new Color(0, 153, 204), 1));
    loginButton = new JButton("Login");
    loginButton.setBackground(new Color(0, 153, 204));
    loginButton.setForeground(Color.WHITE);
    loginButton.setFont(new Font("Arial", Font.BOLD, 16));
    loginButton.addActionListener(new LoginAction());
    // Add components to main panel
    mainPanel.add(titleLabel);
    mainPanel.add(new JLabel()); // Placeholder for layout alignment
    mainPanel.add(roleLabel);
    mainPanel.add(roleComboBox);
    mainPanel.add(usernameLabel);
    mainPanel.add(usernameField);
    mainPanel.add(passwordLabel);
    mainPanel.add(passwordField);
    mainPanel.add(new JLabel()); // Placeholder for layout alignment
    mainPanel.add(loginButton);
    // Add main panel to the background label
    backgroundLabel.add(mainPanel, BorderLayout.CENTER);
    // Add the background label to the frame
    setContentPane(backgroundLabel);
    setVisible(true);
}
private class LoginAction implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        String role = (String) roleComboBox.getSelectedItem();
        String username = usernameField.getText();
        String password = new String(passwordField.getPassword());
```

```
        try (Connection connection = DatabaseConnection.getConnection()) {
            String sql = "SELECT * FROM users WHERE username = ? AND password = ? AND
role = ?";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setString(1, username);
            statement.setString(2, password);
            statement.setString(3, role);
            ResultSet resultSet = statement.executeQuery();
            if (resultSet.next()) {
                if (role.equals("Admin")) {
                    new AdminDashboard(); // Assuming AdminDashboard exists
                } else if (role.equals("Student")) {
                    new StudentDashboard(username); // Pass username to StudentDashboard
                }
                dispose(); // Close login screen
            } else {
                JOptionPane.showMessageDialog(null, "Invalid login credentials.");
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(null, "Error connecting to the database.");
        }
    }
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(LoginScreen::new);
  }
}
```

StudentDashboard code:

```
import javax.swing.*;
import java.awt.*;
class StudentDashboard extends JFrame {
    private String username;
    public StudentDashboard(String username) {
        this.username = username;
        setTitle("Student Dashboard - Elective Course Registration System");
        setSize(600, 400); // Smaller initial size
        setLocationRelativeTo(null); // Center the frame on the screen
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(5, 1, 10, 10)); // Padding between buttons
        // Font and color settings for buttons
        Font buttonFont = new Font("Arial", Font.BOLD, 18); // Reduced font size for compact
look
        Color buttonColor = new Color(106, 226, 222); // Light blue background
        Color textColor = Color.BLACK;
        // Create buttons with consistent styling
        JButton viewCoursesButton = createStyledButton("View Available Courses", buttonFont,
buttonColor, textColor);
        JButton registerCourseButton = createStyledButton("Register for Course", buttonFont,
buttonColor, textColor);
        JButton withdrawCourseButton = createStyledButton("Withdraw from Course",
buttonFont, buttonColor, textColor);
        JButton viewRegisteredCoursesButton = createStyledButton("View Registered Courses",
buttonFont, buttonColor, textColor);
        JButton logoutButton = createStyledButton("Logout", buttonFont, new Color(255, 102,
102), Color.WHITE); // Light red for logout
        // Action listeners for buttons
        viewCoursesButton.addActionListener(e -> new ViewCourseFrame());
        registerCourseButton.addActionListener(e -> new RegisterCourseFrame(username));
        withdrawCourseButton.addActionListener(e -> new WithdrawCourseFrame(username));
        viewRegisteredCoursesButton.addActionListener(e -> new
ViewRegisteredCoursesFrame(username));
        logoutButton.addActionListener(e -> {
            new LoginScreen();
            dispose(); // Close student dashboard
        });

        // Add buttons to the frame
        add(viewCoursesButton);
        add(registerCourseButton);
        add(withdrawCourseButton);
```

```
        add(viewRegisteredCoursesButton);
        add(logoutButton);

        setVisible(true);
    }
    // Helper method to create styled buttons
    private JButton createStyledButton(String text, Font font, Color bgColor, Color textColor) {
        JButton button = new JButton(text);
        button.setFont(font); // Apply compact font size
        button.setBackground(bgColor);
        button.setForeground(textColor);
        button.setFocusPainted(false); // Removes focus border on click
        button.setOpaque(true); // Ensures color is applied on all platforms
        button.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15)); // Padding inside
button
        button.setPreferredSize(new Dimension(200, 40)); // Adjusted button size
        return button;
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new StudentDashboard("student1"));
    }
}
```

AdminDashboard code:

```
import javax.swing.*;
import java.awt.*;
class AdminDashboard extends JFrame {
    public AdminDashboard() {
        setTitle("Admin Dashboard - Elective Course Registration System");
        setSize(600, 400); // Smaller initial size
        setLocationRelativeTo(null); // Center the frame on the screen
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(5, 1, 10, 10)); // Padding between buttons
        // Font and color settings for buttons
        Font buttonFont = new Font("Arial", Font.BOLD, 18); // Reduced font size for compact
look
        Color buttonColor = new Color(106, 226, 222); // Light blue background
        Color textColor = Color.BLACK;
        // Create buttons with consistent styling
        JButton createCourseButton = createStyledButton("Create Course", buttonFont,
buttonColor, textColor);
        JButton deleteCourseButton = createStyledButton("Delete Course", buttonFont,
```

```
buttonColor, textColor);
        JButton editCourseButton = createStyledButton("Edit Course", buttonFont, buttonColor, 
textColor);
        JButton viewCoursesButton = createStyledButton("View All Courses", buttonFont, 
buttonColor, textColor);
        JButton logoutButton = createStyledButton("Logout", buttonFont, new Color(255, 102, 
102), Color.WHITE); // Light red for logout
        // Action listeners for buttons
        createCourseButton.addActionListener(e -> new CreateCourseFrame());
        deleteCourseButton.addActionListener(e -> new DeleteCourseFrame());
        editCourseButton.addActionListener(e -> new EditCourseFrame());
        viewCoursesButton.addActionListener(e -> new ViewCourseFrame());
        logoutButton.addActionListener(e -> {
            new LoginScreen();
            dispose(); // Close admin dashboard
        });
        // Add buttons to the frame
        add(createCourseButton);
        add(deleteCourseButton);
        add(editCourseButton);
        add(viewCoursesButton);
        add(logoutButton);
        setVisible(true);
    }
    // Helper method to create styled buttons
    private JButton createStyledButton(String text, Font font, Color bgColor, Color textColor) {
        JButton button = new JButton(text);
        button.setFont(font); // Apply compact font size
        button.setBackground(bgColor);
        button.setForeground(textColor);
        button.setFocusPainted(false); // Removes focus border on click
        button.setOpaque(true); // Ensures color is applied on all platforms
        button.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15)); // Padding inside 
button
        button.setPreferredSize(new Dimension(200, 40)); // Adjusted button size
        return button;
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(AdminDashboard::new);
    }
}
```

CreateCourseFrame Code:

```
import javax.swing.*;
import java.awt.*;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
class CreateCourseFrame extends JFrame {
    private JTextField courseIdField, courseNameField, instructorField, locationField;
    private JSpinner maxStudentsSpinner, sectionSpinner;
    private JButton createButton;
    public CreateCourseFrame() {
        setTitle("Create Course");
        setSize(600, 400); // Compact frame size
        setLocationRelativeTo(null); // Center on screen
        setLayout(new GridLayout(7, 2, 10, 10)); // Add padding between components
        // Font and color settings
        Font labelFont = new Font("Arial", Font.BOLD, 20); // Consistent font size for labels
        Font fieldFont = new Font("Arial", Font.PLAIN, 18); // Slightly smaller font for input fields
        // Course ID
        JLabel courseIdLabel = new JLabel("Course ID:");
        courseIdLabel.setFont(labelFont);
        courseIdField = new JTextField();
        courseIdField.setFont(fieldFont);
        // Course Name
        JLabel courseNameLabel = new JLabel("Course Name:");
        courseNameLabel.setFont(labelFont);
        courseNameField = new JTextField();
        courseNameField.setFont(fieldFont);
        // Max Students
        JLabel maxStudentsLabel = new JLabel("Max Students:");
        maxStudentsLabel.setFont(labelFont);
        maxStudentsSpinner = new JSpinner(new SpinnerNumberModel(10, 1, 100, 1));
        maxStudentsSpinner.setFont(fieldFont);
        // Instructor
        JLabel instructorLabel = new JLabel("Instructor:");
        instructorLabel.setFont(labelFont);
        instructorField = new JTextField();
        instructorField.setFont(fieldFont);
        // Section
        JLabel sectionLabel = new JLabel("Section:");
        sectionLabel.setFont(labelFont);
        sectionSpinner = new JSpinner(new SpinnerNumberModel(1, 1, 10, 1));
        sectionSpinner.setFont(fieldFont);
```

```
        // Location
        JLabel locationLabel = new JLabel("Location:");
        locationLabel.setFont(labelFont);
        locationField = new JTextField();
        locationField.setFont(fieldFont);
        // Create button
        createButton = new JButton("Create Course");
        createButton.setFont(new Font("Arial", Font.BOLD, 22));
        createButton.setBackground(new Color(0, 153, 204)); // Consistent color with
dashboard button
        createButton.setForeground(Color.WHITE);
        createButton.setFocusPainted(false);
        createButton.setBorder(BorderFactory.createEmptyBorder(10, 15, 10, 15)); // Padding
inside button
        // Action listener for button
        createButton.addActionListener(e -> {
            String courseId = courseIdField.getText();
            String courseName = courseNameField.getText();
            int maxStudents = (int) maxStudentsSpinner.getValue();
            String instructor = instructorField.getText();
            int section = (int) sectionSpinner.getValue();
            String location = locationField.getText();
            if (courseId.isEmpty() || courseName.isEmpty()) {
                JOptionPane.showMessageDialog(this, "Course ID and Course Name cannot be
empty.");
                return;
            }

            try (Connection connection = DatabaseConnection.getConnection()) {
                String sql = "INSERT INTO courses (course_id, course_name, max_students,
instructor, section, location) VALUES (?, ?, ?, ?, ?, ?)";
                PreparedStatement statement = connection.prepareStatement(sql);
                statement.setString(1, courseId);
                statement.setString(2, courseName);
                statement.setInt(3, maxStudents);
                statement.setString(4, instructor);
                statement.setInt(5, section);
                statement.setString(6, location);

                int rowsInserted = statement.executeUpdate();
                if (rowsInserted > 0) {
                    JOptionPane.showMessageDialog(this, "Course created successfully!");
                    dispose(); // Close the frame after successful creation
                }
```

```
        } catch (SQLException ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(this, "Error saving course to the database.");
        }
    });
    // Adding components to the frame
    add(courseIdLabel); add(courseIdField);
    add(courseNameLabel); add(courseNameField);
    add(maxStudentsLabel); add(maxStudentsSpinner);
    add(instructorLabel); add(instructorField);
    add(sectionLabel); add(sectionSpinner);
    add(locationLabel); add(locationField);
    add(new JLabel()); add(createButton); // Empty label for alignment

    setVisible(true);
  }
}
```

RegisteredCourseFrame Code:

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;
public class RegisterCourseFrame extends JFrame {
  private String username;
  private JTextField courseIdField;
  public RegisterCourseFrame(String username) {
    this.username = username;
    setTitle("Register for Course");
    setSize(400, 150); // Increased size for better arrangement
    setLayout(new GridBagLayout());
    setLocationRelativeTo(null);
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(10, 10, 10, 10);  // Padding around components
    Font labelFont = new Font("Arial", Font.PLAIN, 25);
    Font fieldFont = new Font("Arial", Font.PLAIN, 25);
    Color buttonColor = new Color(0, 204, 255);
    // Course ID Label and Field
    JLabel courseIdLabel = new JLabel("Course ID:");
    courseIdLabel.setFont(labelFont);
    courseIdField = new JTextField();
    courseIdField.setFont(fieldFont);
    // Set GridBagLayout for the Course ID Label and Field
```

```
        gbc.gridx = 0; gbc.gridy = 0; gbc.anchor = GridBagConstraints.WEST;
        add(courseIdLabel, gbc);
        gbc.gridx = 1; gbc.gridy = 0; gbc.fill = GridBagConstraints.HORIZONTAL;
        add(courseIdField, gbc);
        // Register Button
        JButton registerButton = new JButton("Register");
        registerButton.setFont(new Font("Arial", Font.PLAIN, 25));
        registerButton.setBackground(buttonColor);
        registerButton.addActionListener(e -> registerCourse());
        // Set GridBagLayout for Register Button
        gbc.gridx = 1; gbc.gridy = 1; gbc.fill = GridBagConstraints.NONE;
        add(registerButton, gbc);
        setVisible(true);
    }
    private void registerCourse() {
        String courseId = courseIdField.getText();
        if (courseId.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please enter a course ID.");
            return;
        }
        try (Connection connection = DatabaseConnection.getConnection()) {
            String sql = "INSERT INTO registrations (username, course_id) VALUES (?, ?)";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setString(1, username);
            statement.setString(2, courseId);

            int rowsInserted = statement.executeUpdate();
            if (rowsInserted > 0) {
                JOptionPane.showMessageDialog(this, "Successfully registered for course " +
courseId);
                dispose();
            }
        } catch (SQLException ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(this, "Error: Could not register for course.");
        }
    }
}
```

ViewCourseFrame Code:

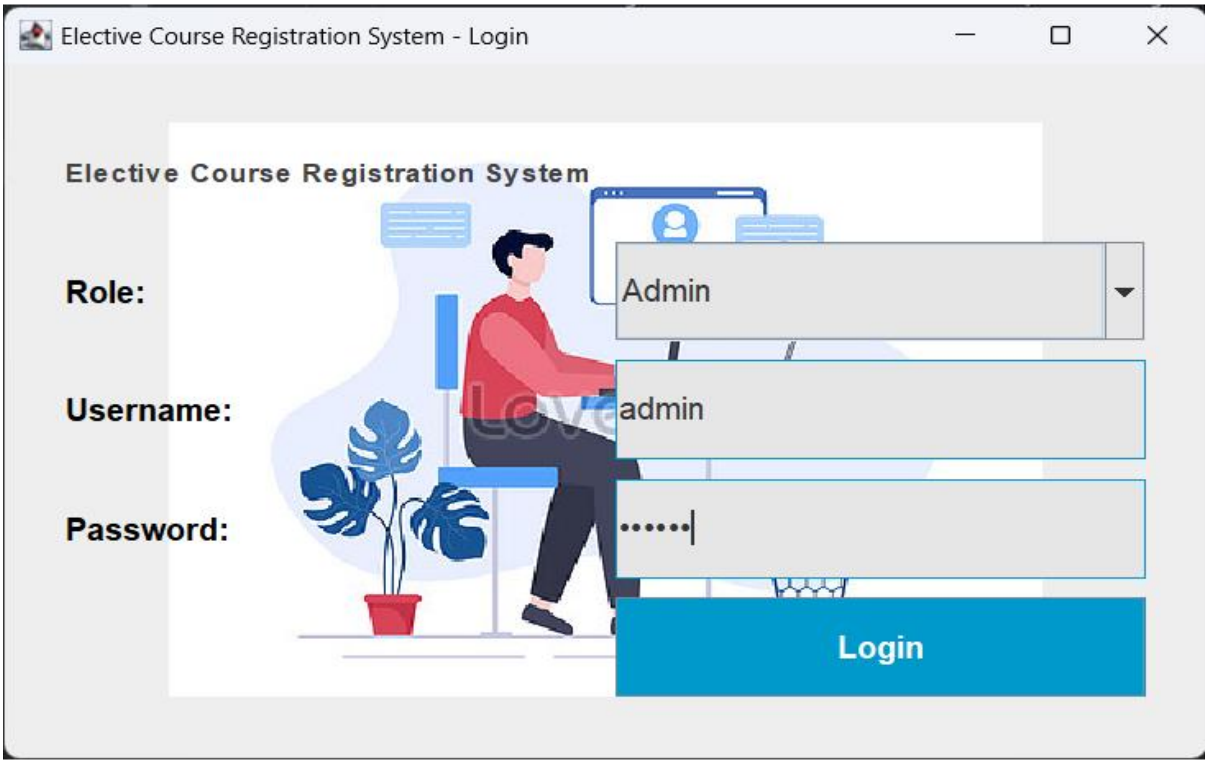
```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
public class ViewCourseFrame extends JFrame {
  private JTable coursesTable;
  private JScrollPane scrollPane;
  public ViewCourseFrame() {
    setTitle("View All Courses");
    setSize(900, 300);
    setLayout(new BorderLayout());
    setLocationRelativeTo(null);
    // Column names for the table
    String[] columnNames = {"Course ID", "Course Name", "Instructor", "Location", "Max
Students", "Section"};
    // Initialize table with no data initially
    DefaultTableModel tableModel = new DefaultTableModel(columnNames, 0);
    coursesTable = new JTable(tableModel);
    coursesTable.setFillsViewportHeight(true);
    // Add table to a scroll pane
    scrollPane = new JScrollPane(coursesTable);
    add(scrollPane, BorderLayout.CENTER);
    // Fetch and populate data from database
    fetchCourseData(tableModel);
    // Add a back button
    JButton backButton = new JButton("Back");
    backButton.addActionListener(e -> dispose()); // Close the current frame and return to
the previous screen
    // Add the back button at the bottom
    JPanel buttonPanel = new JPanel();
    buttonPanel.add(backButton);
    add(buttonPanel, BorderLayout.SOUTH);
    setVisible(true);
  }
  private void fetchCourseData(DefaultTableModel tableModel) {
    String query = "SELECT course_id, course_name, instructor, location, max_students,
section FROM courses";
    try (Connection connection = DatabaseConnection.getConnection();
       PreparedStatement statement = connection.prepareStatement(query);
```

```
        ResultSet resultSet = statement.executeQuery()) {
        // Loop through the result set and add rows to the table model
        while (resultSet.next()) {
            String courseId = resultSet.getString("course_id");
            String courseName = resultSet.getString("course_name");
            String instructor = resultSet.getString("instructor");
            String location = resultSet.getString("location");
            int maxStudents = resultSet.getInt("max_students");
            int section = resultSet.getInt("section");
            // Add row to the table model
            tableModel.addRow(new Object[]{courseId, courseName, instructor, location,
maxStudents, section});
        }
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(this, "Error fetching course data from the
database.");
    }
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(ViewCourseFrame::new);
  }
}
```

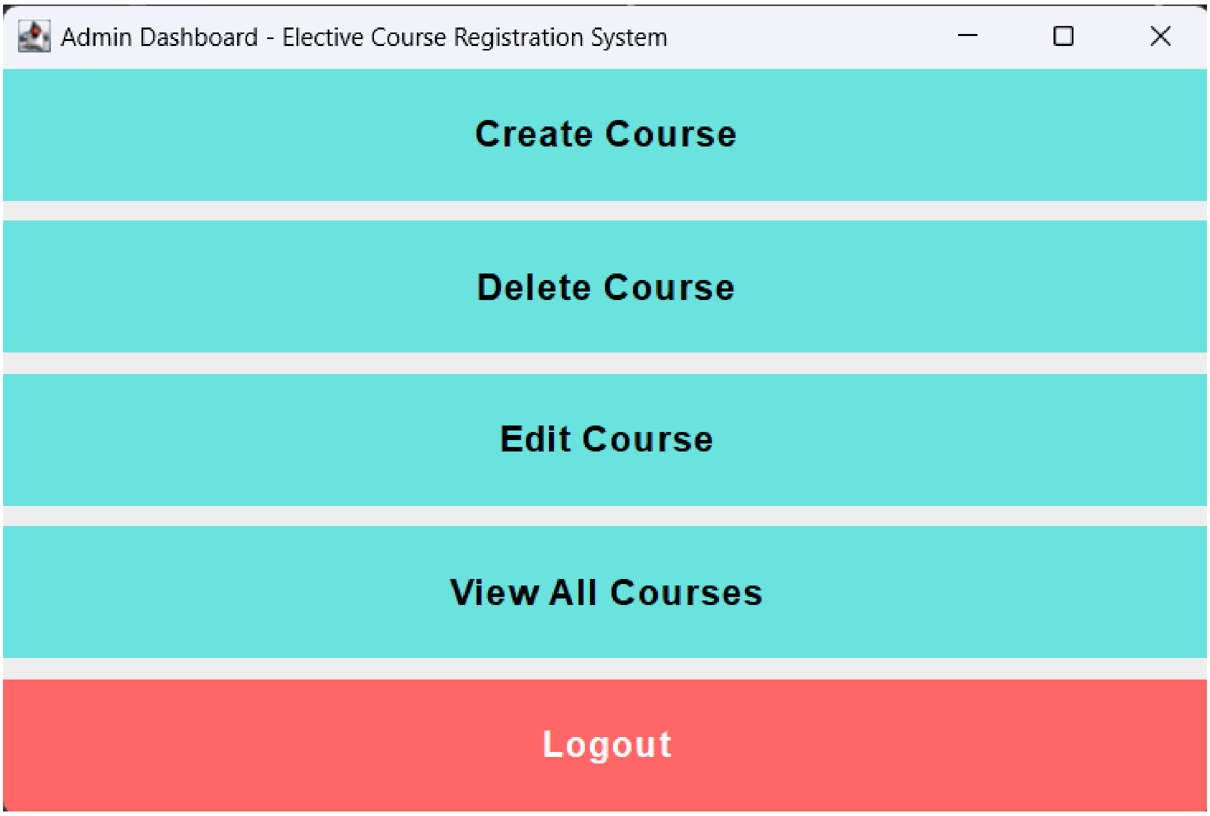
DatabaseConnection Code:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
  public static Connection getConnection() throws SQLException {
    String url = "jdbc:mysql://localhost:3307/course_registration_system";
    String user = "root";
    String password = "root";

    return DriverManager.getConnection(url, user, password);
  }
}
```

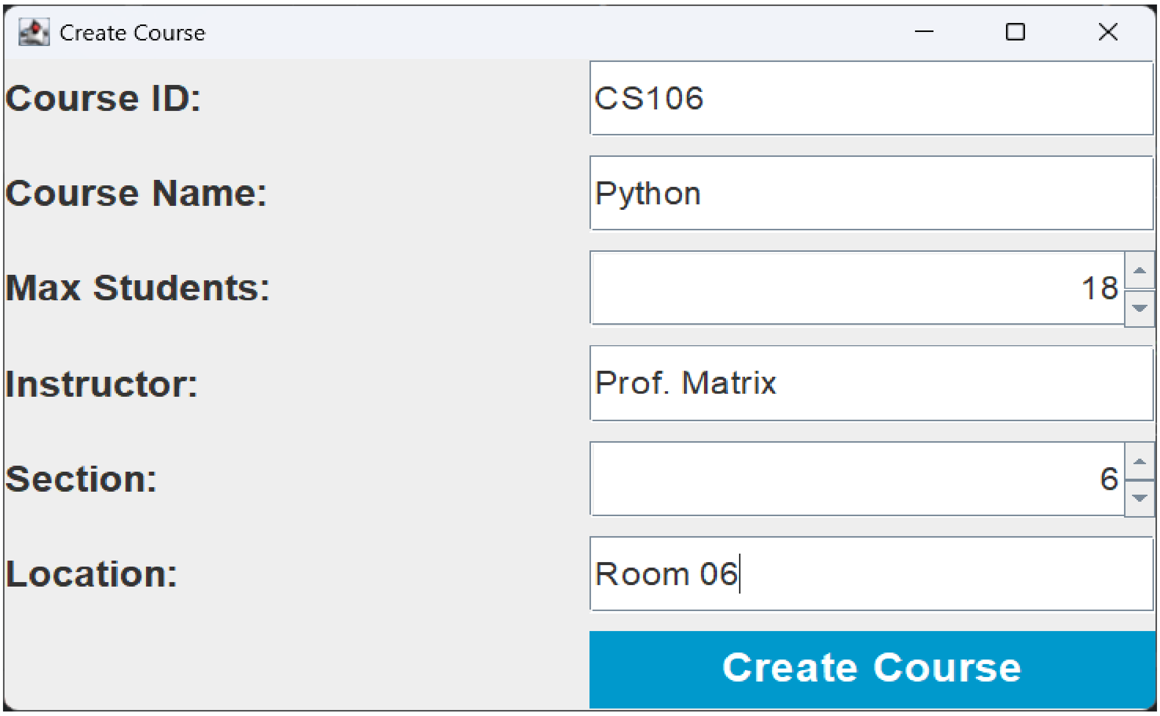
Output:

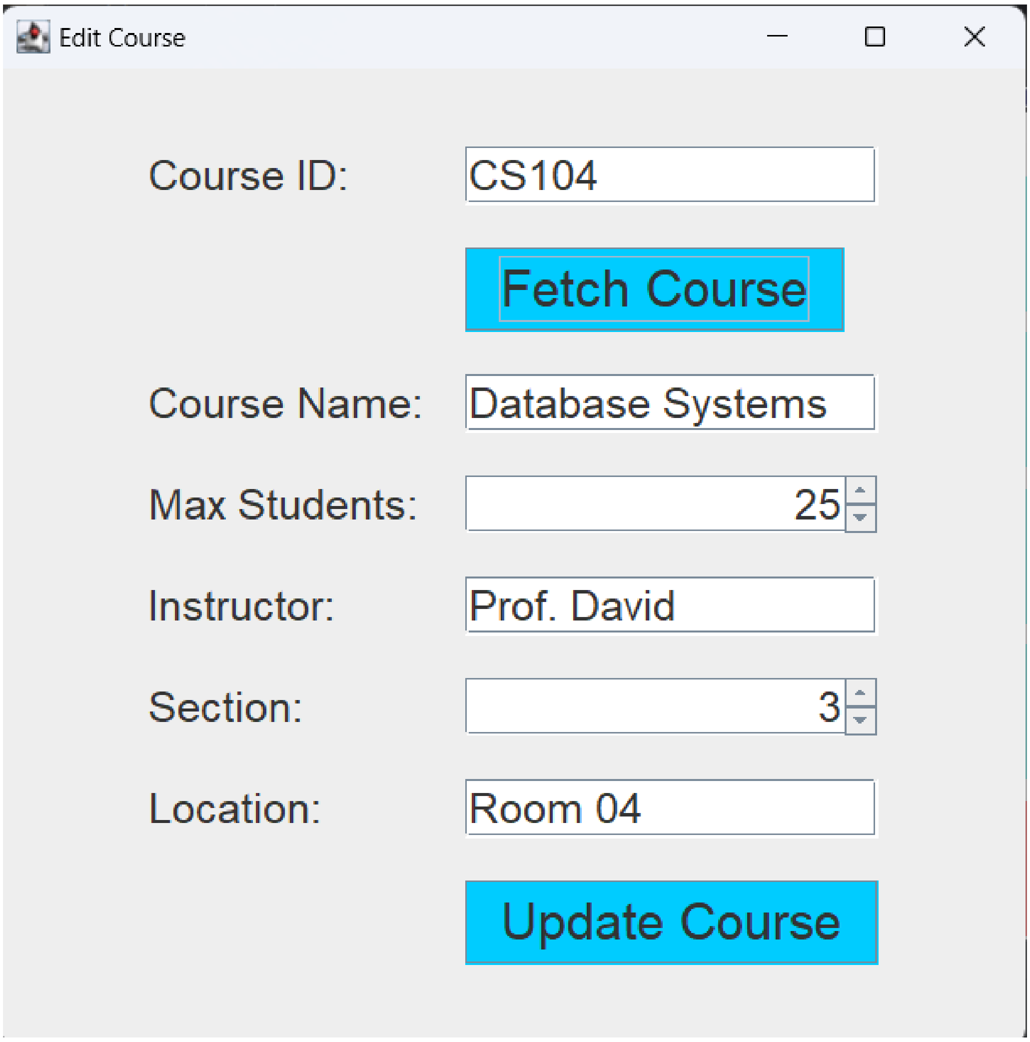


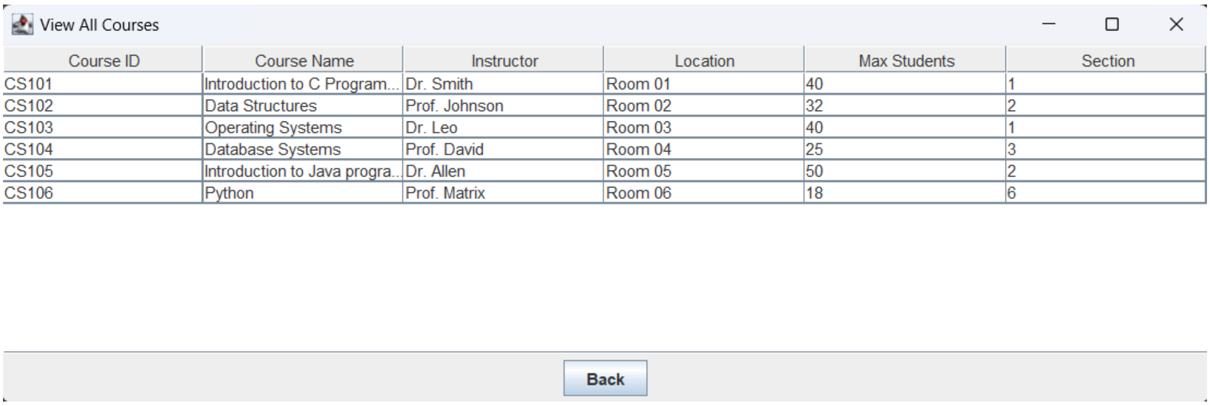
Elective Course Registration System - Login
Elective Course Registration System
Role:
Admin
Username:
admin
Password:
Login



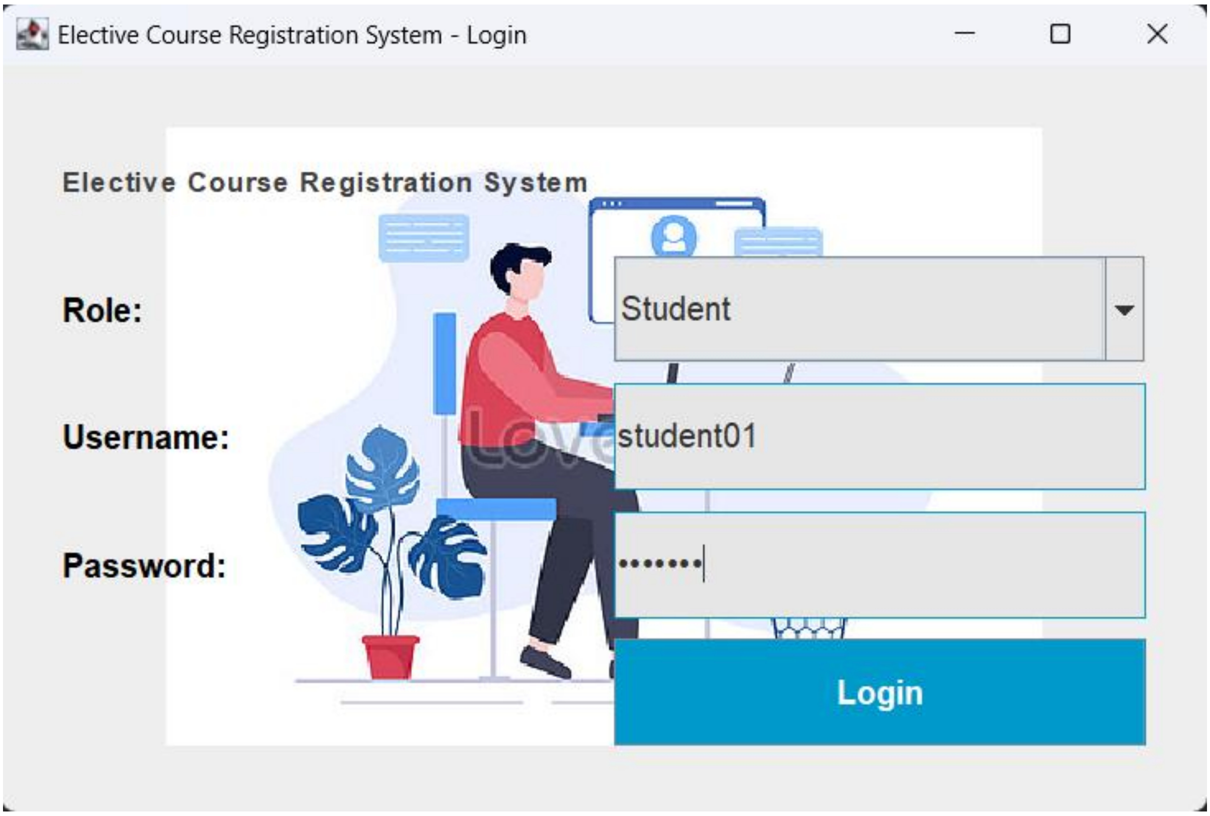
Admin Dashboard - Elective Course Registration System
Create Course
Delete Course
Edit Course
View All Courses
Logout

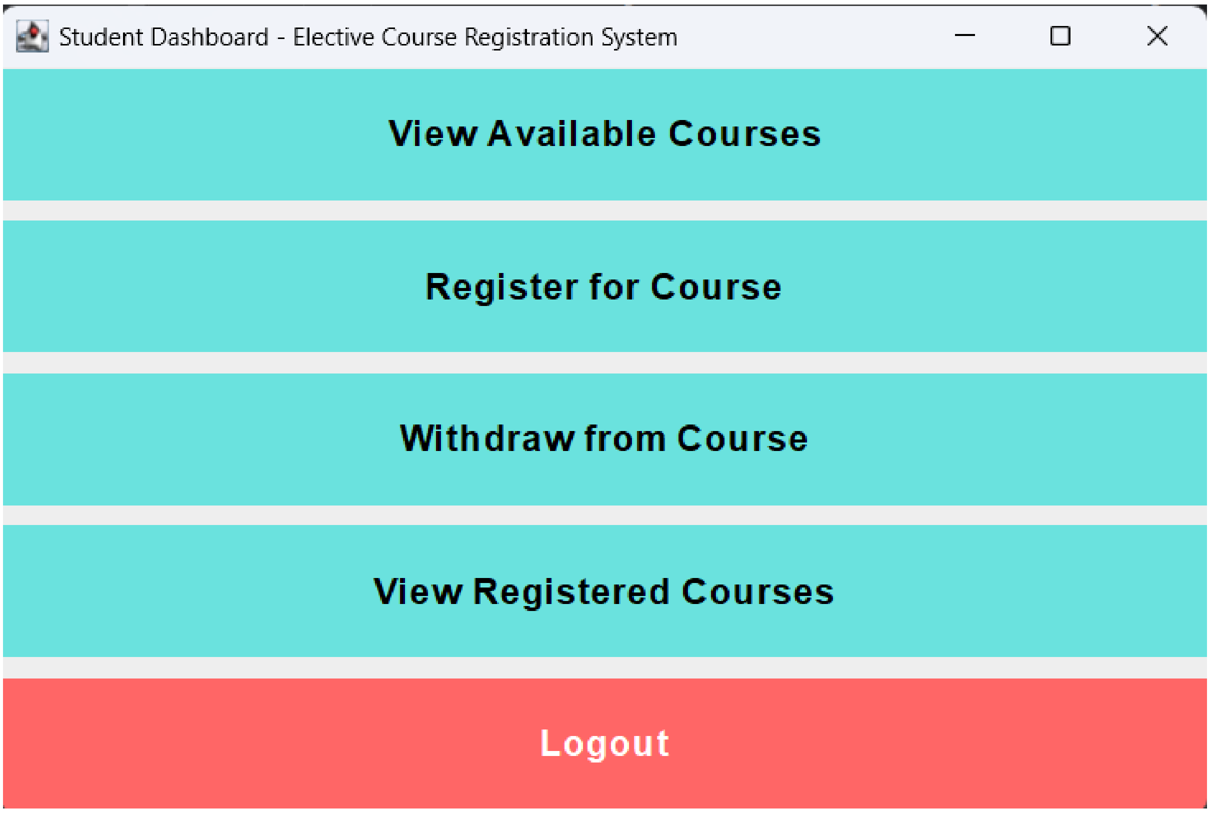

Create Course
Course ID:
CS106
Course Name:
Python
Max Students:
18
Instructor:
Prof. Matrix
Section:
6
Location:
Room 06
Create Course

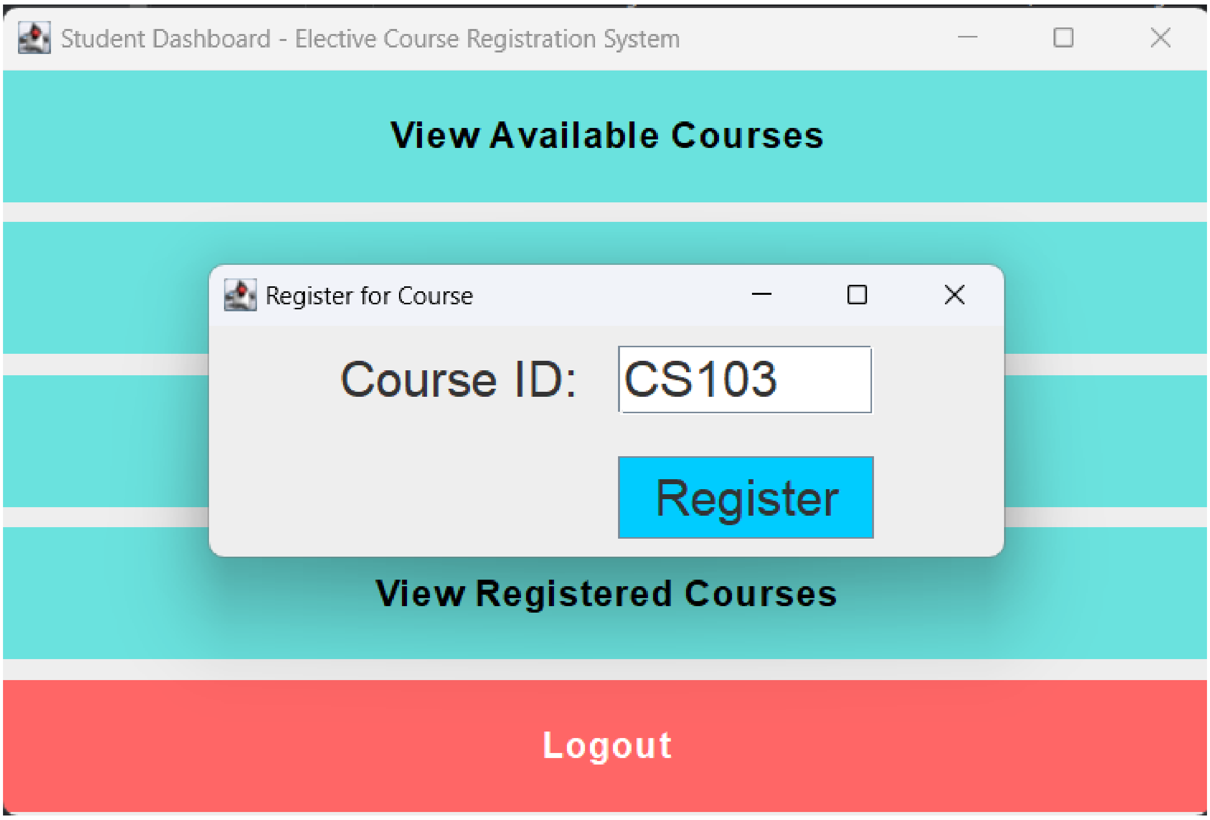

Edit Course
Course ID:
CS104
Fetch Course
Course Name:
Database Systems
Max Students:
25
Instructor:
Prof. David
Section:
3
Location:
Room 04
Update Course

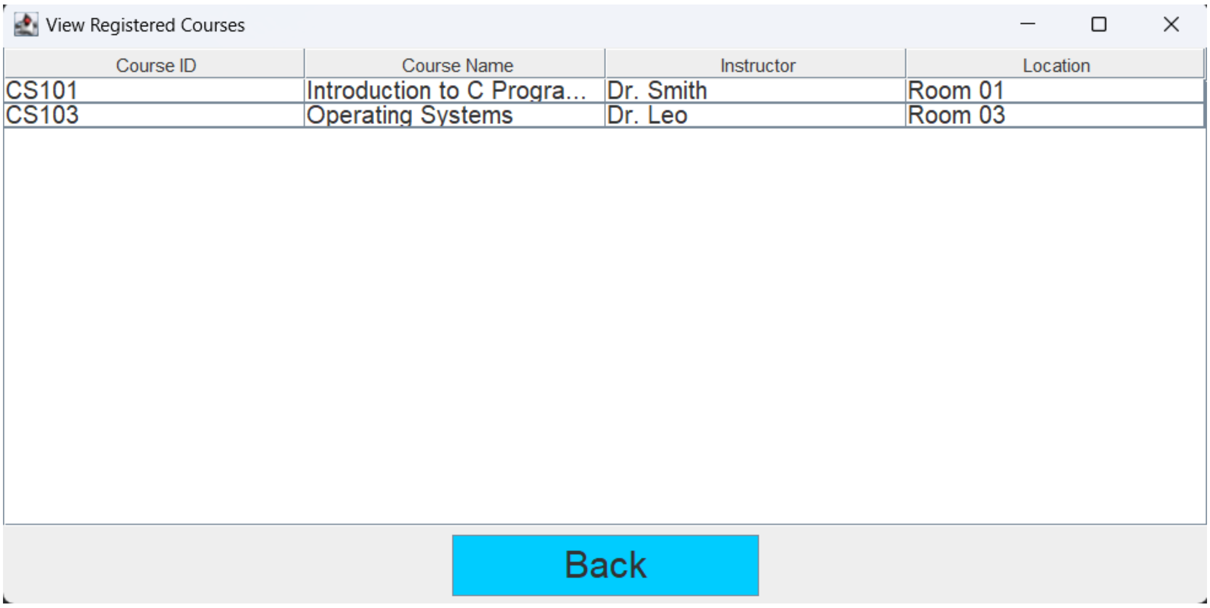


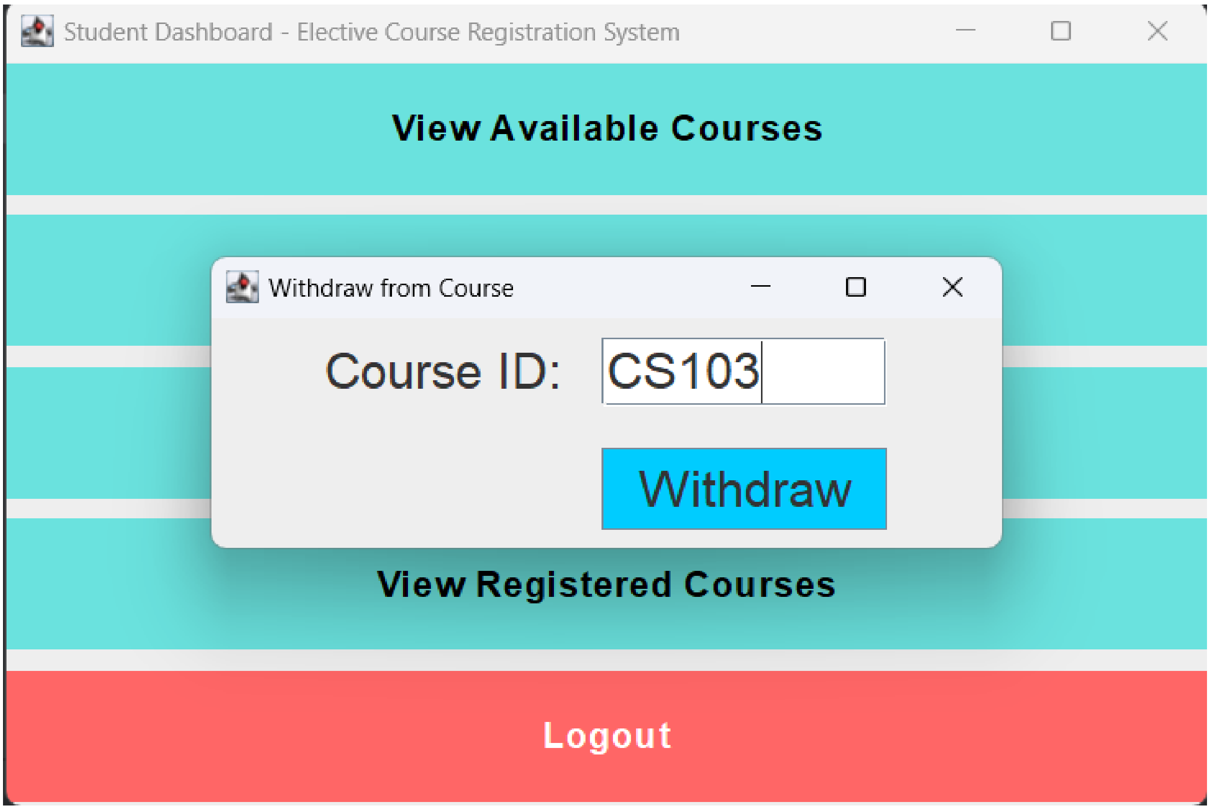
Course ID	Course Name	Instructor	Location	Max Students	Section
CS101	Introduction to C Program...	Dr. Smith	Room 01	40	1
CS102	Data Structures	Prof. Johnson	Room 02	32	2
CS103	Operating Systems	Dr. Leo	Room 03	40	1
CS104	Database Systems	Prof. David	Room 04	25	3
CS105	Introduction to Java progra...	Dr. Allen	Room 05	50	2
CS106	Python	Prof. Matrix	Room 06	18	6

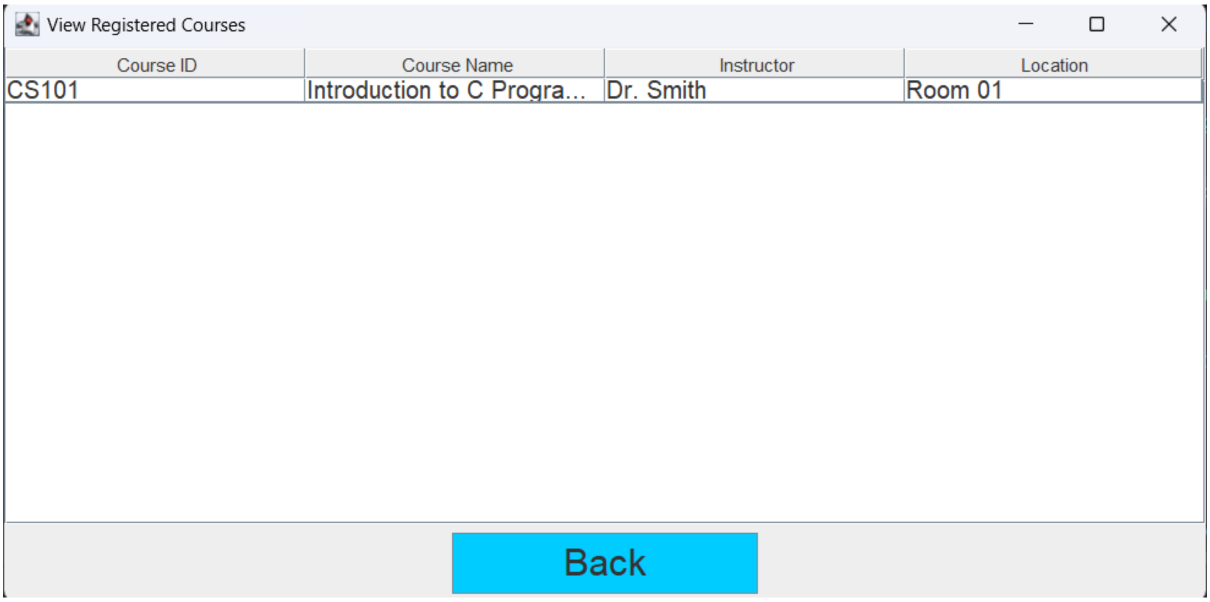



Student Dashboard - Elective Course Registration System
View Available Courses
Register for Course
Withdraw from Course
View Registered Courses
Logout


Student Dashboard - Elective Course Registration System
View Available Courses
Register for Course
Course ID:
CS103
Register
View Registered Courses
Logout


View Registered Courses
Course ID
Course Name
Instructor
Location
CS101
Introduction to C Progra...
Dr. Smith
Room 01
CS103
Operating Systems
Dr. Leo
Room 03
Back


Student Dashboard - Elective Course Registration System
View Available Courses
Withdraw from Course
Course ID:
CS103
Withdraw
View Registered Courses
Logout



MySQL Database: